

Priority Pack

Everything you lost points on in your last exam (27 / 36 needed), fixed

Every Haskell function here was compiled and run with GHC while this PDF was built; the output shown under each `ghci>` line is the real output. Every grammar conversion and CYK table was checked by a program on every string up to length 9–12. You can trust every answer in here.

What you lost points on, and where it is fixed

On the exam	What went wrong	Section
Convert a grammar to Chomsky normal form	botched the steps	1
CYK: demonstrate how a string is parsed, show the table, explain	failed	2
Describe the role of a parser (lexer, parser, interpreter)	incomplete	3
Encapsulation / information hiding	too vague / incomplete	4
Call-by-value vs call-by-reference	(you got points — keep it)	5
Haskell: write <code>foldl</code> ; list $\rightarrow$ integer	could not write it	6
Haskell: write <code>decompress</code>	not answered	6

HOW TO USE THIS PACK (ABOUT 4–5 HOURS OF WORK)

1. Read a section once, slowly. Every section follows the same order: the idea in plain words, the recipe, a fully worked example, then the traps.
2. **Close the PDF and write the model answer or the function from memory on paper.** Then compare. This is the step that matters. Reading alone is how you ended up with 27.
3. Do the **Grill** section (section 7) without looking. The answers are on separate pages after it.
4. Anything you got wrong: redo it tomorrow morning before the exam.

WHERE THE POINTS WERE: 9 MORE WOULD HAVE BEEN A PASS

CNF and CYK are pure **procedures**. There is nothing to be creative about, so a person who follows the recipe step by step gets full points every time. The same goes for the theory questions: they have a fixed “complete answer” (definition + how + example + advantages), and incomplete answers lose half the points. Section 3 and section 4 give you those complete answers.

Contents

1. Chomsky normal form (CNF), step by step

1.1 The recipe (always in this order: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ shape)

1.2 Worked example A — the exam's own grammar
 $S \rightarrow aSbS \mid \lambda$

1.3 Worked example B — every step is needed

1.4 Worked example C — unit chains and useless variables

2. The CYK algorithm: show the table, explain the result

2.1 Worked example 1 — the real exam question (abcab)

2.2 Worked example 2 — a string that is *rejected* (length 5)

2.3 Worked example 3 — accepted, length 6

3. Lexer, parser, interpreter — the complete answer

3.1 Lexer (scanner)

3.2 Parser

3.3 Between parser and executor: static (semantic) analysis

3.4 Executor (the interpreting part)

3.5 See it: a complete tiny interpreter (Haskell, runs)

3.6 Precedence and associativity (asked on both 2023 exams, 6 p)

4. Encapsulation and information hiding — not vague this time

4.1 Definitions

4.2 The same stack, showing both ideas (compiles and runs)

4.3 The neighbours — define these precisely too

5. Parameter passing: by value, by reference (and the others)

5.1 Vocabulary first

5.2 The five methods

5.3 Verified example

6. Haskell: fold, list → int, compress/decompress, reverse

6.1 The five pieces of syntax you need

6.2 Pattern 1 — plain recursion

6.3 Pattern 2 — the accumulator (“carry the answer so far”)

6.4 Writing foldr and foldl yourself

6.5 compress (2023-03-16) and decompress (your last exam)

6.6 Tail-recursive Fibonacci (2017 exam)

7. The Grill — do these on paper, without looking

A. Chomsky normal form

B. CYK

C. Theory — write complete answers (definition, mechanism, example, advantages)

D. Haskell — write these (type signature first!)

E. Grammars (both 2023 exams had these — 20 points)

8. Grill — answers

A. Chomsky normal form

B. CYK

C. Theory

D. Haskell

E. Grammars

1. Chomsky normal form (CNF), step by step

THE IDEA IN PLAIN WORDS

A grammar in CNF is a grammar where **every rule is one of only two boring shapes**:

$$A \rightarrow BC \text{ (exactly two variables) or } A \rightarrow a \text{ (exactly one terminal).}$$

Why would anyone want that? Because then every derivation tree is a **binary tree**, and a string of length n is always derived in exactly $2n - 1$ steps. That makes algorithms like CYK (section 2) possible. Converting to CNF does **not change the language** (except that λ disappears) — it only rewrites the rules into boring shapes.

DEFINITION (WRITE THIS ON THE EXAM)

A context-free grammar is in **Chomsky normal form** if every production has the form

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

where A, B, C are variables and a is a terminal. Every CFG G with $\lambda \notin L(G)$ has an equivalent grammar in CNF. (If $\lambda \in L(G)$, the CNF grammar generates $L(G) \setminus \{\lambda\}$.)

Checking whether a grammar is in CNF

Go through the rules one by one and mark anything that is not of those two shapes:

Rule	CNF?	Why
$S \rightarrow AB$	✓	two variables
$A \rightarrow a$	✓	one terminal
$S \rightarrow aB$	✗	terminal mixed with a variable
$S \rightarrow ABC$	✗	three variables (too long)
$A \rightarrow B$	✗	a <i>unit</i> production (single variable)
$A \rightarrow \lambda$	✗	a λ -production
$A \rightarrow ab$	✗	two terminals

1.1 The recipe (always in this order: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ shape)

RECIPE: CONVERTING TO CNF — ALWAYS IN THIS ORDER

1. Remove λ -productions.

- Find the **nullable** variables: A is nullable if $A \rightarrow \lambda$, or $A \rightarrow B_1 \dots B_k$ where all B_i are nullable. Repeat until nothing new.
- For every rule, add **all versions where you leave out any combination of nullable variables** (but never add $A \rightarrow \lambda$).
- Delete all $A \rightarrow \lambda$.

2. Remove unit productions ($A \rightarrow B$).

- For every A , find all B with $A \Rightarrow^* B$ using *only* unit rules.
- Give A a copy of every **non-unit** rule of each such B .
- Delete all unit rules (including $A \rightarrow A$).

3. Remove useless variables, in this order:

- non-generating**: variables that can never finish into a terminal string (e.g. $C \rightarrow cC$ only);
- non-reachable**: variables that can't be reached from S .

Delete every rule that mentions them.

4. Fix the shapes.

- In every rule whose right side has length ≥ 2 , replace each terminal a by a new variable T_a and add $T_a \rightarrow a$.
- Break every right side with 3 or more variables into a chain: $A \rightarrow C_1 C_2 C_3 \dots C_n$ becomes $A \rightarrow C_1 D_1, D_1 \rightarrow C_2 D_2, \dots, D_{n-2} \rightarrow C_{n-1} C_n$.

WHY THE ORDER MATTERS

Removing λ can **create** unit rules (e.g. $S \rightarrow aSb \mid SS$ with nullable S gives $S \rightarrow S$). Removing units can **create** useless variables (a variable nobody uses any more). So: λ first, units second, useless third, shapes last. If you do it in another order you will have to repeat steps.

1.2 Worked example A — the exam's own grammar $S \rightarrow aSbS \mid \lambda$

This is the grammar for the PL-exam language “equally many a and b , every prefix has at least as many a as b ”. Convert it to CNF.

Step 1 — λ -productions. Nullable: S (because $S \rightarrow \lambda$). The rule $S \rightarrow aSbS$ contains two occurrences of S , so we write all four combinations of keeping/dropping them:

keep 1st S ?	keep 2nd S ?	new rule
yes	yes	$S \rightarrow aSbS$
no	yes	$S \rightarrow abS$
yes	no	$S \rightarrow aSb$
no	no	$S \rightarrow ab$

Delete $S \rightarrow \lambda$. Result: $S \rightarrow aSbS \mid abS \mid aSb \mid ab$.

Step 2 — unit productions. None (no rule of the form $S \rightarrow X$). **Step 3 — useless.** None.

Step 4a — terminals. Introduce $A \rightarrow a$ and $B \rightarrow b$: $S \rightarrow ASBS \mid ABS \mid ASB \mid AB$.

Step 4b — long right sides.

- $S \rightarrow ASBS$ (4 symbols) $\rightarrow S \rightarrow AX$, $X \rightarrow SW$, $W \rightarrow BS$.
- $S \rightarrow ABS$ (3 symbols) $\rightarrow S \rightarrow AY$, $Y \rightarrow BS$.
- $S \rightarrow ASB$ (3 symbols) $\rightarrow S \rightarrow AZ$, $Z \rightarrow SB$.
- $S \rightarrow AB$ is already fine.

FINAL ANSWER

$S \rightarrow AX \mid AY \mid AZ \mid AB$ $X \rightarrow SW$ $W \rightarrow BS$ $Y \rightarrow BS$ $Z \rightarrow SB$ $A \rightarrow a$ $B \rightarrow b$

Since λ was in the original language, the CNF grammar generates the same language minus λ . (You may reuse W instead of introducing Y — both are correct.)

1.3 Worked example B — every step is needed

G : $S \rightarrow ASA \mid aB$ $A \rightarrow B \mid S$ $B \rightarrow b \mid \lambda$

Step 1 — λ -productions.

- Nullable: B (since $B \rightarrow \lambda$); then A (since $A \rightarrow B$ and B is nullable). Is S ? $S \rightarrow ASA$ needs S itself nullable, $S \rightarrow aB$ has a terminal. So **no**. Nullable = $\{A, B\}$.
- $S \rightarrow ASA$: the two A 's may each be dropped: ASA , SA , AS , S .
- $S \rightarrow aB$: drop B : aB , a .
- $A \rightarrow B$: dropping B would give $A \rightarrow \lambda$ — never add that. $A \rightarrow S$ unchanged.
- Delete $B \rightarrow \lambda$.

Result: $S \rightarrow ASA \mid SA \mid AS \mid S \mid aB \mid a$ $A \rightarrow B \mid S$ $B \rightarrow b$

Step 2 — unit productions. The unit rules are $S \rightarrow S$, $A \rightarrow B$, $A \rightarrow S$.

- $S \rightarrow S$: just delete it.
- $A \Rightarrow^* B$ and $A \Rightarrow^* S$. So A gets all non-unit rules of B (b) and of S (ASA , SA , AS , aB , a).

Result: $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$ $A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$ $B \rightarrow b$

Step 3 — useless. Everything generates a terminal string and is reachable. Nothing to remove.

Step 4. $aB \rightarrow UB$ with $U \rightarrow a$. $ASA \rightarrow AX$ with $X \rightarrow SA$.

FINAL ANSWER

$S \rightarrow AX \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AX \mid SA \mid AS \mid UB \mid a$
 $B \rightarrow b$ $U \rightarrow a$ $X \rightarrow SA$

1.4 Worked example C — unit chains and useless variables

G : $S \rightarrow Aa \mid B \mid C$ $A \rightarrow a \mid bA$ $B \rightarrow A \mid bb$ $C \rightarrow cC$

1. λ : no λ -rules. Skip.
2. **Units:** $S \rightarrow B, S \rightarrow C, B \rightarrow A$. Unit reachability: $S \Rightarrow^* B, A, C; B \Rightarrow^* A$. So S gets bb (from B), a, bA (from A), cC (from C); B gets a, bA . Now: $S \rightarrow Aa \mid bb \mid a \mid bA \mid cC, A \rightarrow a \mid bA, B \rightarrow bb \mid a \mid bA, C \rightarrow cC$.
3. **Useless:** (a) C is **not generating** — $C \rightarrow cC$ never ends. Delete C and $S \rightarrow cC$. (b) B is now **unreachable** — nobody mentions B . Delete it. Now: $S \rightarrow Aa \mid bb \mid a \mid bA, A \rightarrow a \mid bA$.
4. **Shapes:** $V \rightarrow a$ and $T \rightarrow b$: $S \rightarrow AV \mid TT \mid a \mid TA, A \rightarrow a \mid TA$.

FINAL ANSWER

$S \rightarrow AV \mid TT \mid a \mid TA \quad A \rightarrow a \mid TA \quad V \rightarrow a \quad T \rightarrow b$

Note: you cannot reuse A for “ $\rightarrow a$ ” here, because A also has the rule $A \rightarrow TA$; a new variable that *only* produces a is required.

THE MISTAKES THAT COST POINTS

- Dropping **only one** combination of nullable symbols. With two nullable symbols in a rule there are up to 4 versions (with three, up to 8). Make the little table.
- Adding $A \rightarrow \lambda$ back by dropping everything.
- Forgetting that a variable is nullable *indirectly* ($A \rightarrow B, B \rightarrow \lambda$).
- Removing a unit rule without copying the rules over (then the language shrinks).
- Replacing a terminal by a variable that has other rules too (see example C).
- Not saying anything about λ . One sentence: “ $\lambda \in L(G)$, so the CNF grammar generates $L(G) - \{\lambda\}$.”

2. The CYK algorithm: show the table, explain the result

THE IDEA IN PLAIN WORDS

Question: *can the grammar produce the string w ?* CYK answers it bottom-up, like building a pyramid:

- First, for every **single letter**: which variables can produce it? (Look for rules $A \rightarrow a$.)
- Then, for every piece of **length 2**: which variables can produce it? Because the grammar is in CNF, a variable produces a piece only through a rule $A \rightarrow BC$: B makes the left part, C makes the right part. So **try every place to cut the piece in two**, and look up the two halves in the rows you already have.
- Then length 3, 4, ... up to the whole string.
- At the top: if the **start symbol** can produce the whole string, $w \in L(G)$.

That's all. The grammar **must be in CNF** first (that's why CNF and CYK always come together).

DEFINITION — WRITE THIS ON THE EXAM

Let $w = a_1 a_2 \cdots a_n$. Define V_{ij} = the set of variables that derive the substring $a_i \cdots a_j$. Then

$$V_{ii} = \{A \mid A \rightarrow a_i\}$$
$$V_{ij} = \bigcup_{k=i}^{j-1} \{A \mid A \rightarrow BC, B \in V_{ik}, C \in V_{k+1,j}\}$$

and $w \in L(G) \iff S \in V_{1n}$. The running time is $O(n^3)$.

RECIPE FOR THE EXAM

1. Check that the grammar is in CNF (say so explicitly — the exam often asks).
2. Draw a triangle: row 1 has n cells (one per letter), row 2 has $n - 1$, ..., the top row has one cell. Label each cell with its substring — this avoids mistakes.
3. Row 1: for each letter, write the variables with a rule $X \rightarrow$ that letter.
4. Row ℓ : for each substring of length ℓ , for **every** cut into (left, right): take the pairs $B \in \text{cell}(\text{left})$, $C \in \text{cell}(\text{right})$, and add every A with a rule $A \rightarrow BC$. Write $\emptyset$ if nothing.
5. Top cell: contains S ? Then write the conclusion sentence (below).

THE “VERY BRIEFLY EXPLAIN WHAT THE RESULT MEANS” SENTENCE

“Cell V_{ij} contains exactly the variables that can derive the substring from position i to j . Since the start symbol S is in the top cell V_{1n} , $S \Rightarrow^* w$, so the string **is** in the language of the grammar.” (If $S \notin V_{1n}$: “...so the string is **not** in the language.”)

Bonus sentence for full points: “Following the entries back down (which cut produced S) gives a derivation tree.”

2.1 Worked example 1 — the real exam question (abcab)

Grammar: $S \rightarrow SS \mid XY \mid XS', S' \rightarrow SY, X \rightarrow a, Y \rightarrow BC \mid BB \mid b, B \rightarrow b, C \rightarrow CC \mid c$.

Is it in CNF? Yes: every rule is either two variables ($SS, XY, XS', SY, BC, BB, CC$) or one terminal (a, b, c). (S' is just a variable name.) So the answer is “Yes”.

Here is every cell with every cut tried — exactly the work you do on paper:

V11 ('a'): variables with a rule $\rightarrow a: \{X\}$
V22 ('b'): variables with a rule $\rightarrow b: \{B, Y\}$
V33 ('c'): variables with a rule $\rightarrow c: \{C\}$
V44 ('a'): variables with a rule $\rightarrow a: \{X\}$
V55 ('b'): variables with a rule $\rightarrow b: \{B, Y\}$

V12 ('ab'): $a|b: \{X\} \cdot \{B, Y\} \rightarrow \{S\} \Rightarrow \{S\}$
V23 ('bc'): $b|c: \{B, Y\} \cdot \{C\} \rightarrow \{Y\} \Rightarrow \{Y\}$
V34 ('ca'): $c|a: \{C\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$
V45 ('ab'): $a|b: \{X\} \cdot \{B, Y\} \rightarrow \{S\} \Rightarrow \{S\}$

V13 ('abc'): $a|bc: \{X\} \cdot \{Y\} \rightarrow \{S\}; ab|c: \{S\} \cdot \{C\} \rightarrow \emptyset \Rightarrow \{S\}$
V24 ('bca'): $b|ca: \{B, Y\} \cdot \emptyset \rightarrow \emptyset; bc|a: \{Y\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$
V35 ('cab'): $c|ab: \{C\} \cdot \{S\} \rightarrow \emptyset; ca|b: \emptyset \cdot \{B, Y\} \rightarrow \emptyset \Rightarrow \emptyset$

V14 ('abca'): $a|bca: \{X\} \cdot \emptyset \rightarrow \emptyset; ab|ca: \{S\} \cdot \emptyset \rightarrow \emptyset; abc|a: \{S\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$
V25 ('bcab'): $b|cab: \{B, Y\} \cdot \emptyset \rightarrow \emptyset; bc|ab: \{Y\} \cdot \{S\} \rightarrow \emptyset; bca|b: \emptyset \cdot \{B, Y\} \rightarrow \emptyset \Rightarrow \emptyset$

V15 ('abcab'): $a|bcab: \{X\} \cdot \emptyset \rightarrow \emptyset; ab|cab: \{S\} \cdot \emptyset \rightarrow \emptyset; abc|ab: \{S\} \cdot \{S\} \rightarrow \{S\}; abca|b: \emptyset \cdot \{B, Y\} \rightarrow \emptyset \Rightarrow \{S\}$

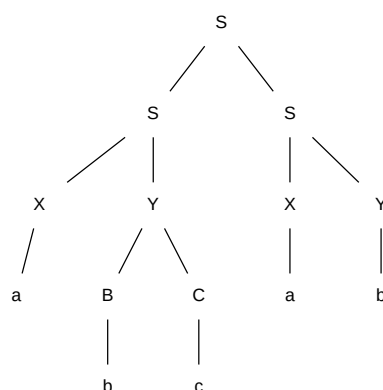
Start symbol $S \in V_{15} \Rightarrow \text{'abcab' IS in } L(G)$

How to read one line: $abc|ab: \{S\} \cdot \{S\} \rightarrow \{S\}$ means “cut *abcab* into *abc* and *ab*; S makes *abc*, S makes *ab*; the rule $S \rightarrow SS$ combines them, so S makes *abcab*”.

The table as you would draw it on the exam (bottom row = single letters):

len 5	$V_{15} = \{S\}$				
len 4	$V_{14} = \emptyset$	$V_{25} = \emptyset$			
len 3	$V_{13} = \{S\}$	$V_{24} = \emptyset$	$V_{35} = \emptyset$		
len 2	$V_{12} = \{S\}$	$V_{23} = \{Y\}$	$V_{34} = \emptyset$	$V_{45} = \{S\}$	
len 1	$V_{11} = \{X\}$	$V_{22} = \{B, Y\}$	$V_{33} = \{C\}$	$V_{44} = \{X\}$	$V_{55} = \{B, Y\}$
	a	b	c	a	b

Result: $S \in V_{15}$, so $abcab \in L(G)$. Reading back: $S \rightarrow SS$ with the first S deriving *abc* (via $S \rightarrow XY$, $X \rightarrow a, Y \rightarrow BC \rightarrow bc$) and the second deriving *ab* (via $S \rightarrow XY$, $X \rightarrow a, Y \rightarrow b$).



Derivation tree recovered from the table

2.2 Worked example 2 — a string that is *rejected* (length 5)

Grammar: the CNF grammar from section 1.2 (for the language “balanced a/b, prefixes have $\geq$ as many a as b”): $S \rightarrow AX \mid AY \mid AZ \mid AB, X \rightarrow SW, W \rightarrow BS, Y \rightarrow BS, Z \rightarrow SB, A \rightarrow a, B \rightarrow b$. String: aabab .

```
V11 ('a'): variables with a rule  $\rightarrow a: \{A\}$ 
V22 ('a'): variables with a rule  $\rightarrow a: \{A\}$ 
V33 ('b'): variables with a rule  $\rightarrow b: \{B\}$ 
V44 ('a'): variables with a rule  $\rightarrow a: \{A\}$ 
V55 ('b'): variables with a rule  $\rightarrow b: \{B\}$ 

V12 ('aa'):  $a|a: \{A\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$ 
V23 ('ab'):  $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$ 
V34 ('ba'):  $b|a: \{B\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$ 
V45 ('ab'):  $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$ 

V13 ('aab'):  $a|ab: \{A\} \cdot \{S\} \rightarrow \emptyset; aa|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$ 
V24 ('aba'):  $a|ba: \{A\} \cdot \emptyset \rightarrow \emptyset; ab|a: \{S\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$ 
V35 ('bab'):  $b|ab: \{B\} \cdot \{S\} \rightarrow \{W, Y\}; ba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{W, Y\}$ 

V14 ('aaba'):  $a|aba: \{A\} \cdot \emptyset \rightarrow \emptyset; aa|ba: \emptyset \cdot \emptyset \rightarrow \emptyset; aab|a: \emptyset \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$ 
V25 ('abab'):  $a|bab: \{A\} \cdot \{W, Y\} \rightarrow \{S\}; ab|ab: \{S\} \cdot \{S\} \rightarrow \emptyset; aba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{S\}$ 

V15 ('aabab'):  $a|aabab: \{A\} \cdot \{S\} \rightarrow \emptyset; aa|bab: \emptyset \cdot \{W, Y\} \rightarrow \emptyset; aab|ab: \emptyset \cdot \{S\} \rightarrow \emptyset; aaba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$ 

Start symbol  $S \notin V_{15} \Rightarrow$  'aabab' is NOT in  $L(G)$ 
```

$S \notin V_{15} = \emptyset$, so aabab is **not** in the language. Sanity check: it has 3 a's and 2 b's, so it can't be balanced.

✓ (Always do such a sanity check if you can!)

2.3 Worked example 3 — accepted, length 6

```

V11 ('a'): variables with a rule → a: {A}
V22 ('b'): variables with a rule → b: {B}
V33 ('a'): variables with a rule → a: {A}
V44 ('a'): variables with a rule → a: {A}
V55 ('b'): variables with a rule → b: {B}
V66 ('b'): variables with a rule → b: {B}

V12 ('ab'): a|b: {A}·{B} → {S}    ⇒ {S}
V23 ('ba'): b|a: {B}·{A} → ∅      ⇒ ∅
V34 ('aa'): a|a: {A}·{A} → ∅      ⇒ ∅
V45 ('ab'): a|b: {A}·{B} → {S}    ⇒ {S}
V56 ('bb'): b|b: {B}·{B} → ∅      ⇒ ∅

V13 ('aba'): a|ba: {A}·∅ → ∅;  ab|a: {S}·{A} → ∅    ⇒ ∅
V24 ('baa'): b|aa: {B}·∅ → ∅;  ba|a: ∅·{A} → ∅    ⇒ ∅
V35 ('aab'): a|ab: {A}·{S} → ∅;  aa|b: ∅·{B} → ∅    ⇒ ∅
V46 ('abb'): a|bb: {A}·∅ → ∅;  ab|b: {S}·{B} → {Z}    ⇒ {Z}

V14 ('abaa'): a|baa: {A}·∅ → ∅;  ab|aa: {S}·∅ → ∅;  aba|a: ∅·{A} → ∅    ⇒ ∅
V25 ('baab'): b|aab: {B}·∅ → ∅;  ba|ab: ∅·{S} → ∅;  baa|b: ∅·{B} → ∅    ⇒ ∅
V36 ('aabb'): a|abb: {A}·{Z} → {S};  aa|bb: ∅·∅ → ∅;  aab|b: ∅·{B} → ∅    ⇒ {S}

V15 ('abaab'): a|baab: {A}·∅ → ∅;  ab|aab: {S}·∅ → ∅;  aba|ab: ∅·{S} → ∅;  abaa|b: ∅·{B} → ∅
⇒ ∅
V26 ('baabb'): b|aabb: {B}·{S} → {W,Y};  ba|abb: ∅·{Z} → ∅;  baa|bb: ∅·∅ → ∅;  baab|b: ∅·{B} → ∅
⇒ {W,Y}

V16 ('abaabb'): a|baabb: {A}·{W,Y} → {S};  ab|aabb: {S}·{S} → ∅;  aba|abb: ∅·{Z} → ∅;
abaa|bb: ∅·∅ → ∅;  abaab|b: ∅·{B} → ∅    ⇒ {S}

Start symbol S ∈ V16 ⇒ 'abaabb' IS in L(G)

```

$S \in V_{16}$: **abaabb** is in the language (it is $ab \cdot aabb$).

THE MISTAKES THAT COST POINTS

- **Not trying every cut.** A cell of length ℓ has $\ell - 1$ cuts. Example 1's top cell has 4 cuts, and only the third one works.
- Order matters: $A \rightarrow BC$ needs B on the **left** part and C on the **right** part.
- Forgetting that a cell can hold **several** variables (like $\{B, Y\}$) — try all of them.
- Not writing the conclusion sentence. “Briefly explain what the result means” is worth points on its own.
- Running CYK on a grammar that is not in CNF — convert first (section 1).

3. Lexer, parser, interpreter — the complete answer

THE IDEA IN PLAIN WORDS

Your program is, to the computer, just a long **string of characters**: `x = 2 + 3 * 4;`. To run it, the language implementation works like reading a sentence in a foreign language:

1. **Lexer (scanner)** — find the *words*. Group characters into meaningful chunks and label them: `x` is an identifier, `=` is an assignment sign, `2` is a number... Throw away spaces and comments.
2. **Parser** — find the *grammar/structure* of the sentence. Check that the words are in a legal order and build a **tree** that shows what belongs to what: the `3 * 4` is done first and then added to `2`.
3. **Executor (interpreter)** — *do what the sentence says*. Walk the tree, compute the values, store `14` in `x`.



The pipeline (Crafting Interpreters calls it “the mountain”)

3.1 Lexer (scanner)

Input	the source code as a string / stream of characters
Output	a list (sequence) of tokens
Task	Group characters into lexemes (the smallest meaningful pieces: <code>while</code> , <code>x1</code> , <code>3.14</code> , <code>>=</code>) and classify each one as a token . A token typically stores: its type (IDENTIFIER, NUMBER, PLUS, keyword WHILE...), the lexeme text, the literal value (e.g. the number 3.14) and the line number (for error messages). It discards whitespace and comments, recognises keywords, and reports lexical errors (an illegal character like <code>#</code> , an unterminated string).
Theory	Tokens are described by regular expressions (identifier = <code>[a-zA-Z_][a-zA-Z0-9_]*</code>), so the lexer is a finite automaton . It follows the “longest match” rule: <code>>=</code> is one token, not <code>></code> then <code>=</code> ; <code>whilex</code> is an identifier, not the keyword <code>while</code> .
Data structures	a character buffer/string with position pointers (start of the current lexeme, current character); a hash table of keywords (reserved words); the output list/array of tokens ; sometimes it inserts identifiers in a symbol table .

3.2 Parser

Input	the list of tokens from the lexer
Output	a syntax tree — usually an abstract syntax tree (AST) — or syntax errors
Task	Check that the token sequence follows the grammar of the language (the syntax), and build a tree that shows the structure of the program: which operands belong to which operator, which statements are inside which loop/block/function. Precedence and associativity are decided here : from <code>2 + 3 * 4</code> the parser builds <code>Add(2, Mul(3, 4))</code> . It reports syntax errors with a position (“expected ‘)’ at line 7”) and tries to recover (skip to the next statement) to report more than one error.

Theory	The syntax is described by a context-free grammar (BNF), because programs have nesting (parentheses, blocks inside blocks) that regular expressions cannot describe. A parser is in effect a pushdown automaton : it needs a stack. Common techniques: recursive descent (top-down: one function per grammar rule; the call stack is the stack), LL, and LR/LALR (bottom-up, e.g. generated by yacc/bison). An ambiguous grammar is a problem for the parser, since one program would get two trees (two meanings).
Data structures	the token list with an index to the current token (lookahead); a stack (explicit in LR parsers, or the recursion call stack in recursive descent); the output tree (nodes = operators, statements; leaves = literals, identifiers); often a parse table (LL/LR).

3.3 Between parser and executor: static (semantic) analysis

Things the grammar cannot check: Is this variable declared? Which declaration does this name refer to (scope resolution)? Do the types fit (in statically typed languages)? This pass walks the tree and uses a **symbol table** (hash maps, one per scope, stacked/chained for nested scopes). It may annotate the tree with the results.

3.4 Executor (the interpreting part)

Input	the (annotated) syntax tree
Output	the program's behaviour: values, output, side effects; or runtime errors
Task	Walks the tree recursively: to evaluate an Add node, evaluate both children then add (post-order). Executes statements in order, handles control flow (if, loops, calls, return, exceptions), looks up and updates variables, calls functions, reports runtime errors (division by zero, type errors in dynamically typed languages, undefined variable).
Data structures	the AST ; an environment = mapping from names to values, implemented as a hash table , with one table per scope linked to its enclosing scope (a chain of hash tables); a call stack of activation records (parameters, local variables, return address) for function calls; a heap for objects, closures, strings. A bytecode interpreter instead uses an array of instructions and a value stack .

INTERPRETER VS COMPILER (OFTEN ASKED TOGETHER)

A **compiler** translates the whole program to another language (e.g. machine code) *before* it runs; the lexer and parser are the same, but the last stage *generates code* instead of executing. An **interpreter** executes the program directly from its tree (or bytecode). Hybrid: compile to bytecode, then interpret it in a virtual machine (Java, Python), possibly with a JIT compiler. Interpreter: easier, portable, good error messages, slower. Compiler: faster execution, errors found before running, less flexible.

3.5 See it: a complete tiny interpreter (Haskell, runs)

For arithmetic like `2 + 3 * 4`. Look at what each stage produces.

Stage 1 — the lexer

```

data Token = TNum Int | TPlus | TMinus | TTimes | TLParen | TRParen
  deriving Show

-- LEXER: characters -> tokens
lexer :: String -> [Token]
lexer [] = []
lexer (c:cs)
  | isSpace c = lexer cs -- skip whitespace
  | isDigit c = TNum (read (c : takeWhile isDigit cs))
                  : lexer (dropWhile isDigit cs)
  | c == '+' = TPlus : lexer cs
  | c == '-' = TMinus : lexer cs
  | c == '*' = TTimes : lexer cs
  | c == '(' = TLParen : lexer cs
  | c == ')' = TRParen : lexer cs
  | otherwise = error ("lexical error: unexpected " ++ [c])

```

```

ghci> lexer "2 + 3 * 4"
[TNum 2,TPlus,TNum 3,TTimes,TNum 4]

```

The syntax-tree type

```

data Expr = Num Int | Add Expr Expr | Sub Expr Expr | Mul Expr Expr
  deriving Show

```

Stage 2 — a recursive-descent parser: one function per grammar rule

```

-- PARSER: tokens -> syntax tree. Grammar (one function per rule):
-- expr -> term { ("+" | "-") term } lowest precedence, left-associative
-- term -> factor { "*" factor }
-- factor -> NUMBER | "(" expr ")" highest precedence
parse :: [Token] -> Expr
parse ts = case expr ts of
  (e, []) -> e
  (_, rest) -> error ("syntax error at " ++ show rest)

expr :: [Token] -> (Expr, [Token])
expr ts = let (t, rest) = term ts in loop t rest
  where
    loop acc (TPlus : r) = let (t, r') = term r in loop (Add acc t) r'
    loop acc (TMinus : r) = let (t, r') = term r in loop (Sub acc t) r'
    loop acc r = (acc, r)

term :: [Token] -> (Expr, [Token])
term ts = let (f, rest) = factor ts in loop f rest
  where
    loop acc (TTimes : r) = let (f, r') = factor r in loop (Mul acc f) r'
    loop acc r = (acc, r)

factor :: [Token] -> (Expr, [Token])
factor (TNum n : r) = (Num n, r)
factor (TLParen : r) = case expr r of
  (e, TRParen : r') -> (e, r')
  _ -> error "syntax error: expected )"
factor ts = error ("syntax error at " ++ show ts)

```

```

ghci> parse (lexer "2 + 3 * 4")
Add (Num 2) (Mul (Num 3) (Num 4))
ghci> parse (lexer "10 - 4 - 3")
Sub (Sub (Num 10) (Num 4)) (Num 3)

```

Look at the trees: $*$ ended up **deeper** than $+$ (higher precedence — done first), and $10 - 4 - 3$ became $(10 - 4) - 3$ (left associative). This is exactly what the grammar levels $\text{expr} \rightarrow \text{term} \rightarrow \text{factor}$ produce.

Stage 3 — the executor walks the tree

```
-- EXECUTOR: walk the tree
eval :: Expr -> Int
eval (Num n)    = n
eval (Add a b)  = eval a + eval b
eval (Sub a b)  = eval a - eval b
eval (Mul a b)  = eval a * eval b

run :: String -> Int
run = eval . parse . lexer
```

```
ghci> run "2 + 3 * 4"
14
ghci> run "10 - 4 - 3"
3
ghci> run "(2 + 3) * 4"
20
```

3.6 Precedence and associativity (asked on both 2023 exams, 6 p)

MODEL ANSWER: “BRIEFLY DESCRIBE PRECEDENCE AND ASSOCIATIVITY. HOW ARE THEY RELATED IN THE CONTEXT OF PROGRAMMING LANGUAGES?”

Precedence decides which of two **different** operators binds tighter when there are no parentheses. $*$ has higher precedence than $+$, so $2 + 3 * 4$ means $2 + (3 * 4) = 14$, not $(2 + 3) * 4 = 20$.

Associativity decides how operators of the **same** precedence level are grouped. $-$ is **left-associative**: $10 - 4 - 3 = (10 - 4) - 3 = 3$, not $10 - (4 - 3) = 9$. Exponentiation and assignment are usually **right-associative**: $2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512$; $a = b = 0 = a = (b = 0)$. Some operators are non-associative ($a < b < c$ is an error in some languages).

Relation: both are rules for resolving the **ambiguity** of expressions written without parentheses, i.e. for deciding the shape of the syntax tree. Precedence is applied first (between levels); associativity is only needed when operators have **the same** precedence. In a grammar they are encoded structurally: one nonterminal per precedence level ($\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$, $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$, $\text{factor} \rightarrow \text{num} \mid (\text{expr})$) — lower levels are deeper in the tree, so they bind tighter; **left recursion** ($\text{expr} \rightarrow \text{expr} + \text{term}$) gives left associativity, **right recursion** ($\text{pow} \rightarrow \text{base} ^ \text{pow}$) gives right associativity. They matter because they change the result ($10-4-3$) and, with side effects, behaviour. Parentheses always override both.

MODEL ANSWER: “DESCRIBE AS MUCH AS THE SPACE PERMITS THE ROLE OF A PARSER IN A PROGRAM”

The parser is the second phase of an interpreter/compiler, after the lexer. Its input is the **sequence of tokens** produced by the lexer (e.g. ID(x) ASSIGN NUM(2) PLUS NUM(3) STAR NUM(4)). Its job is **syntax analysis**: it checks that the token sequence can be derived from the language's **context-free grammar**, and it builds a **syntax tree** (usually an abstract syntax tree) that represents the hierarchical structure of the program — which statements are inside which blocks, and which operands belong to which operators. When building the tree the parser applies the language's **precedence and associativity** rules, so $2+3*4$ becomes $+(2, *(3,4))$. If the tokens do not fit the grammar it reports a **syntax error** with its location and tries to recover to find further errors. The tree is the input to the later phases: semantic analysis (scopes, type checking, with a symbol table) and then execution (interpreter) or code generation (compiler). A common technique is **recursive descent**, where each grammar rule becomes a function and the recursion stack plays the role of the pushdown automaton's stack; other parsers are table-driven (LL, LR). Data structures used: the token list with a current position, a stack (explicit or the call stack), and the tree it outputs.

MODEL ANSWER: “AN INTERPRETER CONSISTS OF A SCANNER/LEXER, A PARSER AND AN EXECUTING PART. WHAT ARE THEIR TASKS AND WHAT DATA STRUCTURES DO THEY USE?” (8 P)

Lexer: reads the source text character by character and groups characters into lexemes, which it classifies as tokens (type, lexeme, value, line). Skips whitespace and comments, distinguishes keywords from identifiers, reports lexical errors. Works like a finite automaton defined by regular expressions. *Data structures:* input string/buffer with position pointers, keyword hash table, output list of tokens.

Parser: takes the token list and checks it against the context-free grammar of the language; builds a syntax tree (AST) that captures structure, precedence and associativity; reports syntax errors. Recursive descent or table-driven (LL/LR). *Data structures:* token list + index, a stack (call stack in recursive descent), the tree (and possibly a symbol table).

Executor: traverses the tree and carries out the computation: evaluates expressions recursively, executes statements, handles control flow and function calls, looks up/updates variables, reports runtime errors. *Data structures:* the AST, environments (hash tables mapping names to values, chained for nested scopes), a call stack of activation records (parameters, locals, return address), and a heap for dynamically allocated data.

4. Encapsulation and information hiding — not vague this time

THE IDEA IN PLAIN WORDS

Think of a **medicine capsule**. **Encapsulation** is putting the ingredients *together in one capsule*: the data and the operations that work on it are packed into one unit (a class, a module, an ADT). **Information hiding** is making the capsule *opaque*: from outside you can't see or touch what is inside; you can only use it the intended way (swallow it = call its public operations).

They are two different things. You can have a capsule you can see through: a Python class groups data and methods (encapsulation) but anyone can still write `s.data = []` (no real hiding).

4.1 Definitions

ENCAPSULATION

An abstract data type is specified as **one syntactic unit** (class, module, package) that contains the **representation** of the data *together with* the **operations** on it. Other parts of the program can use the type **only through that definition**.

Advantages: better **organisation** (everything about stacks is in one place), code is **easier to read and understand**, **easier to modify and maintain**, and units can be **compiled separately**.

INFORMATION HIDING

The **internal representation** of the unit (its data fields, helper functions, implementation details) is **not accessible from outside** the unit. Clients see only the **interface** (the public operations and their types).

Advantages: **reliability** — code that is not “authorised” cannot read or change the internal data, so the unit can guarantee its own **invariants** (e.g. `top` always points at the top element). And **changeability** — the representation can be replaced (array → linked list) without changing any client code, because no client could depend on it. It also lowers what a programmer must know to use the unit.

How languages provide it: access modifiers `private` / `protected` / `public` (Java, C++, C#); separate definition and implementation modules (Modula-2: the definition module is public, the implementation module is hidden); export lists (Haskell modules exporting a type but not its constructor; ML `abstype`).

4.2 The same stack, showing both ideas (compiles and runs)

```
#include <iostream>

class Stack {           // ENCAPSULATION: data + operations in one unit
private:               // INFORMATION HIDING: invisible from outside
    int data[100];
    int top = -1;
public:                // the interface: the only way in
    void push(int x) { if (top < 99) data[++top] = x; }
    int pop()          { return data[top--]; }
    bool isEmpty()     { return top == -1; }
};

int main() {
    Stack s;
    s.push(1); s.push(2); s.push(3);
    while (!s.isEmpty()) std::cout << s.pop() << " ";
    std::cout << "\n";
#ifdef CHEAT
    s.top = 50;         // try to break the stack from outside
#endif
}
```

3 2 1

Now “cheat” by compiling with the line `s.top = 50;` switched on — the compiler refuses. *That* is information hiding: the language enforces it.

error: ‘int Stack::top’ is private within this context

MODEL ANSWER: “EXPLAIN ENCAPSULATION” (WRITE ALL FOUR PARTS)

(1) Definition. Encapsulation means that a data type’s representation and the operations on it are defined together as one syntactic unit (a class, module or package), so the rest of the program uses the type only through that unit’s definition. **(2) Purpose.** It is a way of achieving data abstraction: the user thinks in terms of “a stack with push/pop”, not in terms of arrays and indices. **(3) Example.** A class `Stack` containing the array and the `top` index together with `push`, `pop`, `isEmpty`. **(4) Advantages.** Better organisation of the code, easier to read, understand and modify, and supports separate compilation of the unit. It is often combined with, but is not the same as, information hiding.

MODEL ANSWER: “EXPLAIN INFORMATION HIDING” (WRITE ALL FIVE PARTS)

(1) **Definition.** Information hiding means that the internal representation and implementation details of a unit are not visible or accessible outside it; clients can only use the public interface. (2) **Mechanism.** E.g. private fields with public methods in Java/C++, an export list in a Haskell module that exports the type but not its constructors, or Modula-2's separate definition and implementation modules. (3) **Example.** In the stack class, data and top are private; `s.top = 50` from outside is a compile error, so nobody can put the stack into an inconsistent state. (4) **Advantages.** Reliability/integrity (invariants cannot be broken by outside code), the implementation can be changed without affecting clients (low coupling), and users need to know less. (5) **Relation to encapsulation.** Encapsulation *groups* data and operations; information hiding *restricts access* to the grouped internals. Together they give an abstract data type. (Possible disadvantage to mention: some overhead — calls to getters/setters — and sometimes less flexibility, e.g. a subclass or a trusted piece of code may legitimately need internal access; that is why languages add *protected* or “friend”.)

4.3 The neighbours — define these precisely too

Term	Precise definition	Tiny example
Abstraction	Representing something by its essential properties and ignoring details that are irrelevant at the current level. Two kinds: process abstraction (subprograms: you call <code>sort(list)</code> without knowing the algorithm) and data abstraction (abstract data types: you use a stack through push/pop without knowing its representation). Humans need it to manage complexity.	<code>sort(xs)</code> ; a Stack ADT
Abstract data type (ADT)	A data type whose values can only be created and used through a set of operations, defined by what the operations do (e.g. equations <code>pop(push(s,e)) = s</code> , <code>top(push(s,e)) = e</code>), not how. Implemented with encapsulation + information hiding.	stack: empty, push, pop, top, isEmpty
Inheritance	A class (subclass) is defined as an extension of an existing class (superclass): it inherits its fields and methods and can add new ones or override existing ones. Gives code reuse and an “is-a” hierarchy. Single (Java) or multiple (C++).	<code>class Dog extends Animal</code>
Polymorphism	“Many forms”: one name/interface can be used with values of different types . Subtype polymorphism: a variable of type <code>Animal</code> can hold a <code>Dog</code> , and <code>a.speak()</code> runs the Dog version (via dynamic binding). Parametric : one definition works for all types (generics; Haskell <code>length :: [a] -> Int</code>). Ad hoc : overloading — the same name has different implementations for different types (<code>+</code> on ints and floats; Haskell type classes).	<code>animal.speak()</code> ; <code>length</code> on any list

Dynamic binding	Which method implementation a call runs is decided at run time from the object's actual class (not from the variable's declared type). Flexible but costs a little speed; static binding decides at compile time (faster).	virtual methods in C++, all methods in Java
------------------------	---	---

WHY YOUR ANSWER WAS "TOO VAGUE"

"Encapsulation is hiding the data" is (a) vague and (b) mixes up the two concepts. Examiners want: a precise definition, the mechanism, an example, the advantages, and the difference from the related concept. The model answers above have exactly that structure — use it for *every* "explain the concept X" question.

5. Parameter passing: by value, by reference (and the others)

THE IDEA IN PLAIN WORDS

By value = you give the function a **photocopy**. It can scribble on the copy; your original is untouched. **By reference** = you give the function **the address of your original**. Whatever it writes, it writes on your original.

5.1 Vocabulary first

- **Formal parameter**: the name in the function *definition* (`x` in `void f(int x)`).
- **Actual parameter (argument)**: the expression in the *call* (`a+1` in `f(a+1)`).
- **Modes** (semantic models): **in** (data goes into the function), **out** (data comes back to the caller), **in-out** (both).
- **Implementation**: essentially either **copy the value** or **pass an access path (address)**.

5.2 The five methods

Method	Mode	How it works	Problems
By value	in	The value of the actual parameter is copied into the formal parameter, which acts as a local variable. Changes inside do not affect the caller.	Copying large data (arrays, structs) costs time and memory.
By result	out	No value is passed in. At return, the formal parameter's final value is copied back to the actual parameter (which must be a variable).	<code>procA(a, a)</code> : two results copied to the same variable — the order of copying decides the value. <code>procA(a[i], i)</code> : is the address of <code>a[i]</code> computed at the call or at the return?
By value-result (copy-in/copy-out)	in-out	Copied in at the call and copied back at the return.	Same aliasing/order problems as by result; copying cost twice. <code>procA(4+4)</code> : an expression cannot receive a result.
By reference	in-out	The address of the actual parameter is passed. The formal parameter is an alias of the caller's variable; every read and write goes to the original.	Aliasing (two names for one variable: <code>f(a, a)</code>); unwanted side effects “open up” the caller's data; indirect access is slightly slower.

By name	in-out	The argument expression is passed <i>unevaluated</i> and re-evaluated every time the parameter is used (like textual substitution; Algol 60).	Hard to understand and slow; basis for lazy evaluation (Haskell uses call-by-need).
---------	--------	---	---

5.3 Verified example

```
#include <iostream>
using namespace std;

void swapByValue(int x, int y)    { int t = x; x = y; y = t; }
void swapByReference(int &x, int &y) { int t = x; x = y; y = t; }

void fRef(int &x, int &y) { x = x + 1; y = y * 2; }           // called as fRef(a, a)

int main() {
    int a = 1, b = 2;
    swapByValue(a, b);
    cout << "after swapByValue(a, b):    a = " << a << ", b = " << b << "\n";
    swapByReference(a, b);
    cout << "after swapByReference(a, b): a = " << a << ", b = " << b << "\n";

    int c = 5;
    fRef(c, c);                                           // x and y are ALIASES of c
    cout << "by reference, f(c, c) with c = 5:    c = " << c << "\n";

    // pass-by-value-result, simulated: copy in, work on copies, copy out (left to right)
    int d = 5;
    int x = d, y = d;           // copy in
    x = x + 1; y = y * 2;        // body works on local copies
    d = x; d = y;               // copy out: x first, then y (the last copy wins)
    cout << "by value-result, f(d, d) with d = 5: d = " << d << " (would be " << x
        << " if copied out in the other order)\n";
}
```

```
after swapByValue(a, b):    a = 1, b = 2
after swapByReference(a, b): a = 2, b = 1
by reference, f(c, c) with c = 5:    c = 12
by value-result, f(d, d) with d = 5: d = 10 (would be 6 if copied out in the other order)
```

The last two lines are the aliasing problem in numbers: with the **same** function body and the **same** call `f(c, c)`, pass-by-reference gives 12, pass-by-value-result gives 10 (or 6, depending on copy-back order).

MODEL ANSWER: “EXPLAIN THE DIFFERENCES BETWEEN PASS-BY-VALUE AND PASS-BY-REFERENCE. ADVANTAGES AND DISADVANTAGES?” (6 P)

Pass-by-value: at the call, the *value* of the actual parameter is evaluated and **copied** into the formal parameter, which then behaves like a local variable. Changes to it inside the subprogram do not affect the caller (in-mode). *Advantages:* safe — no side effects on the caller’s variables, easy to reason about; the argument can be any expression ($f(a+1)$); access to the parameter is fast (local). *Disadvantages:* copying large objects costs time and memory; it cannot be used to return results through parameters.

Pass-by-reference: the *address* of the actual parameter is passed, so the formal parameter is an **alias** for the caller’s variable; all reads and writes go directly to the original (in-out mode). *Advantages:* efficient — no copying, only an address is passed, even for large structures; the subprogram can return results through parameters (e.g. `swap(a, b)`). *Disadvantages:* side effects — the caller’s data can be changed unintentionally; **aliasing** (e.g. `f(a, a)` , or an array element and the whole array) makes programs hard to understand and verify; each access goes through an indirection (slightly slower); the argument must be a variable, not an arbitrary expression.

Example: `swap(a, b)` works only with pass-by-reference; with pass-by-value it swaps the copies and `a` , `b` are unchanged. (Java passes everything by value, but for objects the value *is* a reference — so the object can be modified, but the caller’s variable cannot be made to point elsewhere.)

6. Haskell: fold, list → int, compress / decompress, reverse

THE IDEA IN PLAIN WORDS

Haskell has no loops and no variables that change. Instead of “repeat this for each element”, you say: “**what is the answer for the empty list?**” and “**if I already had the answer for the rest of the list, how do I get the answer for the whole list?**”. That’s it. Every exam function is one of **two patterns**: plain recursion, or recursion with an **accumulator**. Learn both patterns and you can write all of them.

6.1 The five pieces of syntax you need

1. type signature (write it FIRST — it tells you what the function takes and returns)

```
countA :: String -> Int
countA [] = 0
countA (c:cs) = if c == 'a' then 1 + countA cs else countA cs
```

- `[]` is the empty list; `(x:xs)` matches a non-empty list: `x` = first element (head), `xs` = the rest (tail). A `String` is a list of `Char`, so `"abc" = 'a' : 'b' : 'c' : []`.
- `x : xs` **puts** an element in front ($O(1)$). `xs ++ ys` **glues** two lists (costs $O(\text{length } xs)$).
- **Guards** — a cleaner if/else:

```
countA :: String -> Int
countA [] = 0
countA (c:cs)
  | c == 'a' = 1 + countA cs
  | otherwise = countA cs
```

- **where** — local helper definitions, written after the function body (indented).
- **Lambda** — an unnamed function: `\x -> x * 2` is “the function that takes `x` and returns `x*2`”. `\acc x -> acc + x` takes two arguments.

6.2 Pattern 1 — plain recursion

RECIPE

1. Write the type signature.
2. Base case: `f [] = ...` — the answer for the empty list.
3. Recursive case: `f (x:xs) = ... x ... f xs ...` — **assume** `f xs` **already works** (“trust the recursion”) and combine it with `x`.
4. Test by hand on a 2–3 element list.

`map`: apply `f` to every element

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = f x : map f xs
```

filter: keep the elements where p is True

```
filter :: (a -> Bool) -> [a] -> [a]
filter p []      = []
filter p (x:xs)
  | p x          = x : filter p xs
  | otherwise    = filter p xs
```

```
ghci> map (*2) [1,2,3]
[2,4,6]
ghci> filter even [1..10]
[2,4,6,8,10]
```

6.3 Pattern 2 — the accumulator (“carry the answer so far”)

INTUITION

Like a loop with a running total: `acc = start; for x in list: acc = update(acc, x); return acc`. In Haskell the “loop” is a helper function `go` that takes the rest of the list and the accumulator. When the list is empty, the accumulator *is* the answer.

The accumulator template

```
template :: [Int] -> Int
template xs = go xs 0           -- 0 = the starting value of the accumulator
  where
    go []      acc = acc         -- list finished: the accumulator is the answer
    go (y:ys) acc = go ys (acc + y) -- update the accumulator, continue with the rest
```

This is also **tail recursion**: the recursive call is the very last thing done, so nothing has to be remembered on the stack (it runs like a loop). That is what “write a tail-recursive function” (2017 exam) asks for.

myRev — the exact exam question (10 p, asked twice)

“Write `myRev` that reverses a list, without built-in functions, in $O(n)$.”

```
myRev :: [a] -> [a]
myRev xs = go xs []
  where
    go []      acc = acc
    go (y:ys) acc = go ys (y : acc)
```

```
ghci> myRev "Hello"
"olleH"
ghci> myRev [1,2,3]
[3,2,1]
```

Trace (do this on the exam!): `go [1,2,3] []` $\rightarrow$ `go [2,3] [1]` $\rightarrow$ `go [3] [2,1]` $\rightarrow$ `go [] [3,2,1]` $\rightarrow$ `[3,2,1]`. Each step does one `:` (constant time), there are n steps $\Rightarrow$ **$O(n)$** . Say that sentence on the exam.

THE $O(N^2)$ TRAP

```
slowRev :: [a] -> [a]
slowRev [] = []
slowRev (x:xs) = slowRev xs ++ [x]      -- correct, but  $O(n^2)$ !
```

This is correct but `xs ++ [x]` walks through all of `xs` every time: $n + (n - 1) + \dots + 1 = O(n^2)$. The exam says “linear”, so this loses points. (It also uses `++`, a library function.)

listToInt — a list of digits to a number

`[1,2,3] → 123`. Think like reading digits left to right on a calculator: every new digit **shifts the number so far one place left** ($\times 10$) and adds the digit.

```
listToInt :: [Int] -> Int
listToInt xs = go xs 0
  where
    go [] acc = acc
    go (d:ds) acc = go ds (acc * 10 + d)
```

```
ghci> listToInt [1,2,3]
123
```

Trace: `go [1,2,3] 0 → go [2,3] 1 (0·10+1) → go [3] 12 (1·10+2) → go [] 123 (12·10+3) → 123`.

Variant: from a String of digits (digitToInt comes from Data.Char)

```
strToInt :: String -> Int
strToInt s = listToInt (map digitToInt s)
```

```
ghci> strToInt "2025"
2025
```

The other direction: `Int -> [Int]`

```
intToList :: Int -> [Int]
intToList n
  | n < 10 = [n]
  | otherwise = intToList (n `div` 10) ++ [n `mod` 10]
```

```
ghci> intToList 4071
[4,0,7,1]
```

6.4 Writing foldr and foldl yourself

INTUITION

A **fold** is just “the accumulator pattern, packaged as a library function”. You give it three things: a **combining function**, a **starting value**, and the **list**. It combines everything into one value.

- **foldl** (left): starts from the **left**, carrying the accumulator: `foldl f z [1,2,3] = f (f (f z 1) 2) 3`. The accumulator is the **first** argument of `f`.
- **foldr** (right): builds from the **right**: `foldr f z [1,2,3] = f 1 (f 2 (f 3 z))`. The element is the **first** argument of `f`.

foldl — it IS the accumulator pattern

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f acc [] = acc
foldl f acc (x:xs) = foldl f (f acc x) xs
```

foldr — it IS plain recursion

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f acc [] = acc
foldr f acc (x:xs) = f x (foldr f acc xs)
```

HOW TO REMEMBER THE DEFINITIONS (REWRITE THEM FROM THIS)

- **Both:** on the empty list, return the starting value `acc`.
- **foldl:** “update the accumulator with x, then continue with the rest”: `foldl f (f acc x) xs`. The type: `(b -> a -> b) -> b -> [a] -> b` — f takes **acc first, element second**.
- **foldr:** “combine x with the result for the rest”: `f x (foldr f acc xs)`. The type: `(a -> b -> b) -> b -> [a] -> b` — f takes **element first, acc second**.

Where they differ — subtraction (not commutative):

```
ghci> foldl (-) 0 [1,2,3]
-6
ghci> foldr (-) 0 [1,2,3]
2
```

foldl (-) 0 [1,2,3]	foldr (-) 0 [1,2,3]
<code>= foldl (-) (0-1) [2,3]</code>	<code>= 1 - foldr (-) 0 [2,3]</code>
<code>= foldl (-) ((0-1)-2) [3]</code>	<code>= 1 - (2 - foldr (-) 0 [3])</code>
<code>= foldl (-) (((0-1)-2)-3) []</code>	<code>= 1 - (2 - (3 - foldr (-) 0 []))</code>
<code>= ((0-1)-2)-3 = -6</code>	<code>= 1 - (2 - (3 - 0)) = 2</code>

Using folds (both are one-liners)

```
sum :: [Int] -> Int
sum xs = foldl (+) 0 xs

length :: [a] -> Int
length xs = foldr (\_ n -> n + 1) 0 xs
```

myRev with foldl (only if library functions are allowed)

```
myRevFold :: [a] -> [a]
myRevFold xs = foldl (\acc x -> x : acc) [] xs
```

```
listToIntFold :: [Int] -> Int
listToIntFold = foldl (\acc d -> acc * 10 + d) 0
```

```
ghci> sum [1..10]
55
ghci> length "hello"
5
ghci> myRevFold [1,2,3]
[3,2,1]
ghci> listToIntFold [4,0,7]
407
```

The old exam question with filter and foldl (2023-05-05, 10 p)

```
foo x y = filter (\w -> x == w) y
bar x y = foldl (\w -> \q -> (if q /= x then w + 1 else w)) 0 y
```

foo x y keeps the elements of list **y** that are equal to **x** (so it returns all occurrences of **x**). **bar x y** folds from the left with accumulator **w** starting at 0, and for each element **q** adds 1 if **q /= x**: it **counts the elements that are not equal to x**.

```
ghci> foo 'c' "aabcccaacbbba"
"cccc"
ghci> bar 'c' "aabcccaacbbba"
8
```

Check: the string has 12 characters, 4 of them are 'c' → foo gives "cccc", bar gives 12 – 4 = 8.

6.5 compress (2023-03-16) and decompress (your last exam)

`compress "aaaabbcccb"` should give `[('a',4),('b',2),('c',3),('b',1)]` (run-length encoding). `decompress` goes back.

compress with `takeWhile` / `dropWhile` (library functions allowed)

```
compress :: String -> [(Char, Int)]
compress [] = []
compress (c:cs) = (c, 1 + length same) : compress rest
  where
    same = takeWhile (== c) cs
    rest = dropWhile (== c) cs
```

Read it as: take the first char **c**; **same** = the run of equal characters right after it; **rest** = everything after the run. Output the pair, then compress the rest.

compress with only recursion (no library functions): accumulator = current char and its count

```
compress2 :: String -> [(Char, Int)]
compress2 [] = []
compress2 (c:cs) = go c 1 cs
  where
    go cur n [] = [(cur, n)]
    go cur n (x:xs)
      | x == cur = go cur (n + 1) xs
      | otherwise = (cur, n) : go x 1 xs
```

```
ghci> compress "aaaabbcccb"
[('a',4),('b',2),('c',3),('b',1)]
ghci> compress2 "aaaabbcccb"
[('a',4),('b',2),('c',3),('b',1)]
```

decompress — each pair (c, n) becomes n copies of c

```
decompress :: [(Char, Int)] -> String
decompress [] = []
decompress ((c, n) : ps) = replicate n c ++ decompress ps
```

decompress without `replicate`/`++` (count down)

```
decompress2 :: [(Char, Int)] -> String
decompress2 [] = []
decompress2 ((c, 0) : ps) = decompress2 ps
decompress2 ((c, n) : ps) = c : decompress2 ((c, n - 1) : ps)
```

```
ghci> decompress [('a',4),('b',2),('c',3),('b',1)]
"aaaabbcccb"
ghci> decompress2 (compress "aaaabbcccb")
"aaaabbcccb"
```

If the compressed form is a **string** like "a4b2c3" instead (read the question!):

```
-- "a4b2c3" -> "aaaabbccc" (counts may have several digits: "x12")
decompressStr :: String -> String
decompressStr [] = []
decompressStr (c:cs) = replicate (strToInt digits) c ++ decompressStr rest
  where
    digits = takeWhile isDigit cs
    rest   = dropWhile isDigit cs
```

```
ghci> decompressStr "a4b2c3x12"
"aaaabbccccxxxxxxxxxxxxx"
```

6.6 Tail-recursive Fibonacci (2017 exam)

“Write a tail-recursive function that returns the first X Fibonacci numbers: `fibonacci 10` → `[0,1,1,2,3,5,8,13,21,34]`.” The accumulators are the **two latest numbers** `a` and `b`; each step shifts them: `(a, b) → (b, a+b)`.

```
fibs :: Int -> [Int]
fibs n = go n 0 1
  where
    go 0 _ _ = []
    go k a b = a : go (k - 1) b (a + b)

fibTail :: Int -> Int -- the n-th Fibonacci number, tail recursive
fibTail n = go n 0 1
  where
    go 0 a _ = a
    go k a b = go (k - 1) b (a + b)
```

```
ghci> fibs 10
[0,1,1,2,3,5,8,13,21,34]
ghci> fibTail 10
55
```

THE HASKELL MISTAKES THAT COST POINTS

- No type signature — write it first; it forces you to know what goes in and out.
- Forgetting the base case `f [] = ...` (the function crashes on the empty list).
- Writing `x ++ xs` when `x` is an element — use `x : xs`; `++` needs two **lists** (`[x] ++ xs`).
- `f x:xs` means `(f x) : xs`. In patterns always write parentheses: `f (x:xs)`.
- `foldl`'s function takes `acc` first; `foldr`'s takes the element first. Getting it backwards is a type error (or a wrong answer).
- Indentation: the lines under `where` must line up.

7. The Grill — do these on paper, without looking

Set a timer. Write full answers as if on the exam. Answers start on the next page, so don't turn it until you are done. Mark each question ✓ or ✗ and redo every ✗ tomorrow morning.

A. Chomsky normal form

1. Which rules break CNF, and why? $S \rightarrow AB \mid aB \mid a$ $A \rightarrow ABA \mid b$ $B \rightarrow B \mid \lambda \mid bb$
2. Convert to CNF: $S \rightarrow aSa \mid bSb \mid \lambda$ (even-length palindromes).
3. Convert to CNF: $S \rightarrow AB \mid aB$ $A \rightarrow aab \mid \lambda$ $B \rightarrow bbA$
4. Convert to CNF: $S \rightarrow A \mid aSb$ $A \rightarrow bA \mid b$
5. In example B of section 1.3, why is S *not* nullable even though A is?

B. CYK

6. Using your CNF grammar from question 2, run CYK on `abba` and on `abab`. Show the tables and explain the results.
7. Using the exam grammar $S \rightarrow SS \mid XY \mid XS'$, $S' \rightarrow SY$, $X \rightarrow a$, $Y \rightarrow BC \mid BB \mid b$, $B \rightarrow b$, $C \rightarrow CC \mid c$: run CYK on `aabbb` and on `abcca`.
8. How many cuts do you try for a cell of length 4? For the top cell of a string of length 6?

C. Theory — write complete answers (definition, mechanism, example, advantages)

9. Explain **encapsulation**.
10. Explain **information hiding**, and how it differs from encapsulation.
11. What is **abstraction**? Name its two kinds.
12. What is **polymorphism**? Name the three kinds with an example each.
13. What does the **lexer** do, what is its output, and what data structures does it use?
14. Describe the role of the **parser** in as much detail as you can.
15. What does the **executing part** of an interpreter do and what data structures does it use?
16. Explain **precedence** and **associativity** and how they are related. How does a grammar encode them?
17. Pass-by-value vs pass-by-reference: differences, advantages, disadvantages. What does `f(a, a)` have to do with it?

D. Haskell — write these (type signature first!)

18. `foldl` and `foldr`, from scratch.
19. `myRev` in $O(n)$, without library functions. Explain why it is $O(n)$.
20. `listToInt :: [Int] -> Int` (`[1,2,3] -> 123`), once with an accumulator and once with `foldl`.
21. `compress :: String -> [(Char, Int)]` and `decompress :: [(Char, Int)] -> String`.
22. `countOcc :: Eq a => a -> [a] -> Int` — how many times does an element occur?
23. `myLength`, tail-recursive.

24. `map` and `filter`, written using `foldr`.
25. `sumSqOdd`: the sum of the squares of the odd numbers in a list, using `filter`, `map`, `foldl`. What is `sumSqOdd [1,2,3,4,5]`?
26. `myLast`, `myMaximum`, `elemAt` (the n -th element, counting from 0).
27. `isPal` (is a list a palindrome?) and `dedup` (remove *consecutive* duplicates: `[1,1,2,3,3,3,1] → [1,2,3,1]`).
28. `myZip`, `myAppend` (your own `++`), `myReplicate`.
29. Without a computer, what do these give?
 - a. `foldr (\x acc -> x : x : acc) [] [1,2,3]`
 - b. `foldl (\acc x -> x : acc) [] "abc"`
 - c. `foldr (\x acc -> acc + 1) 0 "hello"`
 - d. `foldl (\acc x -> acc * 2 + x) 0 [1,0,1,1]`
 - e. `foldl (-) 10 [1,2,3]` and `foldr (-) 10 [1,2,3]`

E. Grammars (both 2023 exams had these — 20 points)

30. Write a CFG for “equally many a and b , and every prefix has at least as many a as b ”.
31. Write a CFG for $\{a^n b^m \mid n \neq m\}$.

8. Grill — answers

A. Chomsky normal form

1. $S \rightarrow AB$ ✓, $S \rightarrow a$ ✓, $A \rightarrow b$ ✓. Broken: $S \rightarrow aB$ (terminal together with a variable), $A \rightarrow ABA$ (three variables), $B \rightarrow B$ (unit production — and pointless), $B \rightarrow \lambda$ (λ -production), $B \rightarrow bb$ (two terminals).

2. Nullable: $\{S\}$. Remove λ : $S \rightarrow aSa \mid aa \mid bSb \mid bb$. No units, nothing useless. Terminals $A \rightarrow a$, $B \rightarrow b$: $S \rightarrow ASA \mid AA \mid BSB \mid BB$. Break the long ones:

ANSWER 2

$S \rightarrow AX \mid BY \mid AA \mid BB$ $X \rightarrow SA$ $Y \rightarrow SB$ $A \rightarrow a$ $B \rightarrow b$ (generates the language minus λ)

3. Nullable: $\{A\}$ (not S , not B : both rules of each contain something non-nullable). Remove λ : $S \rightarrow AB \mid B \mid aB$, $A \rightarrow aab$, $B \rightarrow bbA \mid bb$. Units: $S \rightarrow B \rightarrow S$ gets bbA , bb ; delete it: $S \rightarrow AB \mid aB \mid bbA \mid bb$. Useless: none. Shapes with $U \rightarrow a$, $T \rightarrow b$: $S \rightarrow AB \mid UB \mid TTA \mid TT$, $A \rightarrow UUT$, $B \rightarrow TTA \mid TT$; break TTA with $W \rightarrow TA$ and UUT with $V \rightarrow UT$:

ANSWER 3

$S \rightarrow AB \mid UB \mid TW \mid TT$ $A \rightarrow UV$ $V \rightarrow UT$ $B \rightarrow TW \mid TT$ $W \rightarrow TA$ $U \rightarrow a$ $T \rightarrow b$

The classic slip: forgetting to copy bbA into S when removing $S \rightarrow B$.

4. No λ . Unit $S \rightarrow A$: copy bA , b into S : $S \rightarrow aSb \mid bA \mid b$, $A \rightarrow bA \mid b$. Shapes with $U \rightarrow a$, $T \rightarrow b$, $X \rightarrow ST$:

ANSWER 4

$S \rightarrow UX \mid TA \mid b$ $X \rightarrow ST$ $A \rightarrow TA \mid b$ $U \rightarrow a$ $T \rightarrow b$

5. $S \rightarrow ASA$ could only vanish if S itself were already nullable (circular — never established), and $S \rightarrow aB$ contains the terminal a , which can never vanish.

B. CYK

6.

```

V11 ('a'): variables with a rule → a: {A}
V22 ('b'): variables with a rule → b: {B}
V33 ('b'): variables with a rule → b: {B}
V44 ('a'): variables with a rule → a: {A}

V12 ('ab'): a|b: {A}·{B} → ∅ ⇒ ∅
V23 ('bb'): b|b: {B}·{B} → {S} ⇒ {S}
V34 ('ba'): b|a: {B}·{A} → ∅ ⇒ ∅

V13 ('abb'): a|bb: {A}·{S} → ∅; ab|b: ∅·{B} → ∅ ⇒ ∅
V24 ('bba'): b|ba: {B}·∅ → ∅; bb|a: {S}·{A} → {X} ⇒ {X}

V14 ('abba'): a|bba: {A}·{X} → {S}; ab|ba: ∅·∅ → ∅; abb|a: ∅·{A} → ∅ ⇒ {S}

Start symbol S ∈ V14 ⇒ 'abba' IS in L(G)

```

```

V11 ('a'): variables with a rule → a: {A}
V22 ('b'): variables with a rule → b: {B}
V33 ('a'): variables with a rule → a: {A}
V44 ('b'): variables with a rule → b: {B}

V12 ('ab'): a|b: {A}·{B} → ∅ ⇒ ∅
V23 ('ba'): b|a: {B}·{A} → ∅ ⇒ ∅
V34 ('ab'): a|b: {A}·{B} → ∅ ⇒ ∅

V13 ('aba'): a|ba: {A}·∅ → ∅; ab|a: ∅·{A} → ∅ ⇒ ∅
V24 ('bab'): b|ab: {B}·∅ → ∅; ba|b: ∅·{B} → ∅ ⇒ ∅

V14 ('abab'): a|bab: {A}·∅ → ∅; ab|ab: ∅·∅ → ∅; aba|b: ∅·{B} → ∅ ⇒ ∅

Start symbol S ∉ V14 ⇒ 'abab' is NOT in L(G)

```

abba is an even palindrome and is accepted; abab is not a palindrome and is rejected — the sanity check agrees.

7.

```

V11 ('a'): variables with a rule → a: {X}
V22 ('a'): variables with a rule → a: {X}
V33 ('b'): variables with a rule → b: {B,Y}
V44 ('b'): variables with a rule → b: {B,Y}
V55 ('b'): variables with a rule → b: {B,Y}

V12 ('aa'): a|a: {X}·{X} → ∅ ⇒ ∅
V23 ('ab'): a|b: {X}·{B,Y} → {S} ⇒ {S}
V34 ('bb'): b|b: {B,Y}·{B,Y} → {Y} ⇒ {Y}
V45 ('bb'): b|b: {B,Y}·{B,Y} → {Y} ⇒ {Y}

V13 ('aab'): a|ab: {X}·{S} → ∅; aa|b: ∅·{B,Y} → ∅ ⇒ ∅
V24 ('abb'): a|bb: {X}·{Y} → {S}; ab|b: {S}·{B,Y} → {S'} ⇒ {S,S'}
V35 ('bbb'): b|bb: {B,Y}·{Y} → ∅; bb|b: {Y}·{B,Y} → ∅ ⇒ ∅

V14 ('aabb'): a|abb: {X}·{S,S'} → {S}; aa|bb: ∅·{Y} → ∅; aab|b: ∅·{B,Y} → ∅ ⇒ {S}
V25 ('abbb'): a|bbb: {X}·∅ → ∅; ab|bb: {S}·{Y} → {S'}; abb|b: {S,S'}·{B,Y} → {S'} ⇒ {S'}

V15 ('aabbb'): a|abbb: {X}·{S'} → {S}; aa|bbb: ∅·∅ → ∅; aab|bb: ∅·{Y} → ∅; aabb|b: {S}·{B,Y} → {S'} ⇒ {S,S'}

Start symbol S ∈ V15 ⇒ 'aabbb' IS in L(G)

```

V11 ('a'): variables with a rule $\rightarrow a: \{X\}$
V22 ('b'): variables with a rule $\rightarrow b: \{B, Y\}$
V33 ('c'): variables with a rule $\rightarrow c: \{C\}$
V44 ('c'): variables with a rule $\rightarrow c: \{C\}$
V55 ('a'): variables with a rule $\rightarrow a: \{X\}$

V12 ('ab'): $a|b: \{X\} \cdot \{B, Y\} \rightarrow \{S\} \Rightarrow \{S\}$
V23 ('bc'): $b|c: \{B, Y\} \cdot \{C\} \rightarrow \{Y\} \Rightarrow \{Y\}$
V34 ('cc'): $c|c: \{C\} \cdot \{C\} \rightarrow \{C\} \Rightarrow \{C\}$
V45 ('ca'): $c|a: \{C\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$

V13 ('abc'): $a|bc: \{X\} \cdot \{Y\} \rightarrow \{S\}; ab|c: \{S\} \cdot \{C\} \rightarrow \emptyset \Rightarrow \{S\}$
V24 ('bcc'): $b|cc: \{B, Y\} \cdot \{C\} \rightarrow \{Y\}; bc|c: \{Y\} \cdot \{C\} \rightarrow \emptyset \Rightarrow \{Y\}$
V35 ('cca'): $c|ca: \{C\} \cdot \emptyset \rightarrow \emptyset; cc|a: \{C\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$

V14 ('abcc'): $a|bcc: \{X\} \cdot \{Y\} \rightarrow \{S\}; ab|cc: \{S\} \cdot \{C\} \rightarrow \emptyset; abc|c: \{S\} \cdot \{C\} \rightarrow \emptyset \Rightarrow \{S\}$
V25 ('bccca'): $b|cca: \{B, Y\} \cdot \emptyset \rightarrow \emptyset; bc|ca: \{Y\} \cdot \emptyset \rightarrow \emptyset; bcc|a: \{Y\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$

V15 ('abcca'): $a|bccca: \{X\} \cdot \emptyset \rightarrow \emptyset; ab|cca: \{S\} \cdot \emptyset \rightarrow \emptyset; abc|ca: \{S\} \cdot \emptyset \rightarrow \emptyset; abcc|a: \{S\} \cdot \{X\} \rightarrow \emptyset \Rightarrow \emptyset$

Start symbol $S \notin V15 \Rightarrow$ 'abcca' is NOT in $L(G)$

8. A cell of length ℓ has $\ell - 1$ cuts: 3 for length 4, and 5 for the top cell of a length-6 string.

C. Theory

Compare your answers with the model answers — check that you have **every part**:

Q	Must contain	Model answer in
9	one syntactic unit with data + operations; used only through its definition; organisation, readability, modifiability, separate compilation; example	§4.2
10	internal representation not accessible outside; mechanism (private / export list / definition vs implementation module); reliability, invariants, change implementation without affecting clients; example <code>s.top = 50</code> is an error; difference: grouping vs restricting access	§4.2
11	keep essential properties, ignore irrelevant details; process abstraction (subprograms) and data abstraction (ADTs)	§4.3
12	one interface, many types; subtype (inheritance + dynamic binding: <code>animal.speak()</code>), parametric (generics, <code>length :: [a] -> Int</code>), ad hoc (overloading, <code>+</code> on <code>Int</code> and <code>Double</code>)	§4.3
13	characters $\rightarrow$ lexemes $\rightarrow$ tokens (type, lexeme, value, line); skip whitespace/comments; keywords vs identifiers; lexical errors; regular expressions / finite automaton; buffer + positions, keyword hash table, token list	§3.1
14	tokens in; checks against the CFG; builds AST; precedence/associativity; syntax errors + recovery; recursive descent / LL / LR; stack (PDA); output feeds semantic analysis and execution	§3.2, §3.6
15	walks the AST, evaluates recursively, control flow, calls, runtime errors; environments (chained hash tables), call stack of activation records, heap	§3.4
16	precedence = different operators; associativity = same level; both resolve ambiguity / tree shape; grammar levels; left recursion $\Rightarrow$ left assoc; examples <code>2+3*4</code> , <code>10-4-3</code> , <code>2^3^2</code>	§3.6

17	copy vs address/alias; in vs in-out; pros and cons of each; <code>swap</code> ; <code>f(a,a)</code> = aliasing (by reference 12 vs value-result 10 in the example)	§5.3
----	--	------

D. Haskell

18–21: see section 6 (`foldl`, `foldr`, `myRev`, `listToInt`, `compress`, `decompress`). Compare line by line.

22

```
countOcc :: Eq a => a -> [a] -> Int
countOcc _ [] = 0
countOcc y (x:xs)
  | x == y    = 1 + countOcc y xs
  | otherwise = countOcc y xs
```

23

```
myLength :: [a] -> Int
myLength xs = go xs 0
  where
    go []      acc = acc
    go (_:ys) acc = go ys (acc + 1)
```

24

```
mapF :: (a -> b) -> [a] -> [b]
mapF f xs = foldr (\x acc -> f x : acc) [] xs

filterF :: (a -> Bool) -> [a] -> [a]
filterF p xs = foldr (\x acc -> if p x then x : acc else acc) [] xs
```

25

```
sumSqOdd :: [Int] -> Int
sumSqOdd xs = foldl (+) 0 (map (\x -> x * x) (filter odd xs))
```

26

```
myLast :: [a] -> a
myLast [x] = x
myLast (_:xs) = myLast xs

myMaximum :: [Int] -> Int
myMaximum [x] = x
myMaximum (x:xs) = max x (myMaximum xs)

elemAt :: [a] -> Int -> a
elemAt (x:_) 0 = x
elemAt (_:xs) n = elemAt xs (n - 1)
```

27 (isPal)

```
isPal :: Eq a => [a] -> Bool
isPal xs = xs == rev xs []
  where
    rev []      acc = acc
    rev (y:ys) acc = rev ys (y : acc)
```

27 (dedup)

```
dedup :: Eq a => [a] -> [a]
dedup (x:y:rest)
  | x == y    = dedup (y : rest)
  | otherwise = x : dedup (y : rest)
dedup xs = xs           -- 0 or 1 element left
```

28

```
myZip :: [a] -> [b] -> [(a, b)]
myZip (x:xs) (y:ys) = (x, y) : myZip xs ys
myZip _ _          = []

myAppend :: [a] -> [a] -> [a]
myAppend [] ys = ys
myAppend (x:xs) ys = x : myAppend xs ys
```

```
myReplicate :: Int -> a -> [a]
myReplicate 0 _ = []
myReplicate n x = x : myReplicate (n - 1) x
```

```
ghci> countOcc 'a' "banana"
3
ghci> myLength [5,6,7,8]
4
ghci> mapF (+1) [1,2,3]
[2,3,4]
ghci> filterF (>2) [1,5,2,7]
[5,7]
ghci> sumSqOdd [1,2,3,4,5]
35
ghci> myLast [1,2,3]
3
ghci> myMaximum [3,9,2]
9
ghci> elemAt "hello" 1
'e'
ghci> isPal "racecar"
True
ghci> dedup [1,1,2,3,3,3,1]
[1,2,3,1]
ghci> myZip [1,2,3] "ab"
[(1,'a'),(2,'b')]
ghci> myAppend [1,2] [3,4]
[1,2,3,4]
ghci> myReplicate 3 'x'
"xxx"
```

29.

```

ghci> foldr (\x acc -> x : x : acc) [] [1,2,3]
[1,1,2,2,3,3]
ghci> foldl (\acc x -> x : acc) [] "abc"
"cba"
ghci> foldr (\x acc -> acc + 1) 0 "hello"
5
ghci> foldl (\acc x -> acc * 2 + x) 0 [1,0,1,1]
11
ghci> foldl (-) 10 [1,2,3]
4
ghci> foldr (-) 10 [1,2,3]
-8

```

(d) is binary $\rightarrow$ decimal: $1011_2 = 11$. (e) foldl: $((10-1)-2)-3 = 4$; foldr: $1-(2-(3-10)) = 1-9 = -8$.

E. Grammars

30

$S \rightarrow aSbS \mid \lambda$. Why: a non-empty string of this language starts with an a ; match it with the b that first brings the count back to zero. Between them is a balanced string (S), after the b is another balanced string (S). Test: $ab: S \Rightarrow aSbS \Rightarrow abS \Rightarrow ab \checkmark$; $aababbab \checkmark$; ba cannot start $\checkmark$. (Also correct: $S \rightarrow SS \mid aSb \mid \lambda$.)

31

$S \rightarrow aSb \mid A \mid B \quad A \rightarrow aA \mid a \quad B \rightarrow bB \mid b$. Why: aSb builds the equal part from the outside in; in the middle we must put **at least one** extra a (A) or at least one extra b (B). λ is correctly excluded since A, B never give λ .